

```

#include <ESP32Servo.h>      // Gestion des servos sur ESP32
#include <ArduinoJson.h>     // Parsing JSON reçu du Raspberry

Servo esc;      // Servo ESC (traction)
Servo servo;    // Servo direction

// --- ÉTAT ACTUEL ---
// Valeurs réellement envoyées aux servos (après lissage)
int currentServoUS = 1450;
int targetServoUS = 1450;
int currentSpeedUS = 1500;
int targetSpeedUS = 1500;

// --- LIMITES SERVO ---
// Pour éviter que le servo force en butée mécanique
const int SERVO_MIN = 1200;
const int SERVO_MAX = 1650;
const int SERVO_NEUTRE = 1450;

// --- LISSEMENT ---
// Lissage progressif pour éviter les à-coups
const int SMOOTH_STEP = 10;      // Incrément en microsecondes
const int SMOOTH_DELAY = 5;      // Temps entre deux incréments
unsigned long lastSmoothTime = 0;

// --- COOLDOWN ---
// Pour éviter de traiter trop de commandes trop vite
unsigned long lastCmdTime = 0;
const unsigned long CMD_INTERVAL = 40;

// --- FILTRE ANGLE ---
// Pour éviter que le servo bouge pour rien
int lastAngle = 9999;
const int ANGLE_THRESHOLD = 3;

void setup() {
    Serial.begin(115200);      // Communication USB avec le Raspberry

    servo.attach(13);          // Servo direction sur GPIO 13
    esc.attach(15);            // ESC sur GPIO 15

    esc.writeMicroseconds(1500); // Neutre ESC
    delay(3000);               // Temps d'armement ESC

    servo.writeMicroseconds(SERVO_NEUTRE); // Direction centrée

    Serial.println("ESP32 READY (MODE F/R)");
}

void updateSmooth() {
    // Lissage non bloquant basé sur millis()
    unsigned long now = millis();
    if (now - lastSmoothTime < SMOOTH_DELAY) return;
    lastSmoothTime = now;

    // --- LISSEMENT DIRECTION ---
    if (currentServoUS < targetServoUS) currentServoUS += SMOOTH_STEP;
    else if (currentServoUS > targetServoUS) currentServoUS -= SMOOTH_STEP;

    currentServoUS = constrain(currentServoUS, SERVO_MIN, SERVO_MAX);
    servo.writeMicroseconds(currentServoUS);

    // --- LISSEMENT VITESSE ---
    if (currentSpeedUS < targetSpeedUS) currentSpeedUS += SMOOTH_STEP;

```

```

else if (currentSpeedUS > targetSpeedUS) currentSpeedUS -= SMOOTH_STEP;

// Sécurité : on limite la plage PWM
currentSpeedUS = constrain(currentSpeedUS, 1200, 1800);

esc.writeMicroseconds(currentSpeedUS);
}

void loop() {
    updateSmooth();    // Toujours lisser avant de lire les commandes

    if (!Serial.available()) return;    // Rien reçu → on attend

    // Lecture d'une ligne JSON
    String jsonStr = Serial.readStringUntil('\n');
    jsonStr.trim();
    if (jsonStr.length() == 0) return;

    // Cooldown pour éviter surcharge
    unsigned long now = millis();
    if (now - lastCmdTime < CMD_INTERVAL) return;
    lastCmdTime = now;

    // Parsing JSON
    StaticJsonDocument<256> doc;
    if (deserializeJson(doc, jsonStr)) {
        Serial.println("JSON ERROR");
        return;
    }

    const char* action = doc["action"];

    // --- COMMANDE DE CONDUITE ---
    if (strcmp(action, "drive") == 0) {
        int angle = doc["angle"];    // Angle envoyé par le Raspberry
        int speed = doc["speed"];    // PWM vitesse envoyé par le Raspberry

        // MODE F/R : on applique directement la vitesse
        targetSpeedUS = speed;

        // --- TRAITEMENT ANGLE ---
        int a = angle % 360;        // Normalisation 0-359
        if (a > 180) a -= 360;      // Convertit en -180 à +180

        // On ne met à jour que si l'angle change suffisamment
        if (abs(a - lastAngle) >= ANGLE_THRESHOLD) {
            lastAngle = a;

            // Conversion angle → PWM servo
            int limited = constrain(a, -180, 180);
            targetServoUS = map(limited, -180, 180, SERVO_MAX, SERVO_MIN);
        }

        // --- DEBUG ---
        Serial.print("CMD angle=");
        Serial.print(angle);
        Serial.print(" servoUS=");
        Serial.print(targetServoUS);
        Serial.print(" speed=");
        Serial.print(speed);
        Serial.print(" currentSpeed=");
        Serial.println(currentSpeedUS);
    }
}

```

```
// --- COMMANDE STOP ---  
else if (strcmp(action, "stop") == 0) {  
    targetSpeedUS = 1500;    // Neutre  
    Serial.println("STOP CMD");  
}  
}
```